

ci-ext

extension/ci-ext.sh

Revision: 1 **Last modified:** 2026-06-10T15:30:00Z

Overview

`ci-ext.sh` is the **manual** pre-release gate for the BobaLink (Boba/BobaLink) browser extension. It is the **ONLY** release-readiness gate for the extension — there is no CI/CD pipeline and none may be created (Constitution Hard Stop §1 + project `CLAUDE.md` “CI IS MANUAL — permanent”). The operator runs it by hand before packaging/distributing the extension (Phase 9).

It runs, in order, FAILing loudly on the first failing step, and prints CI-EXT: PASS only if every step succeeds:

1. **Type gate** — `npx tsc --noEmit`
2. **Lint gate** — `npm run lint` (eslint over `src/` + `tests/`)
3. **Unit suite** — `npx vitest run` (full suite; requires exit 0, no hardcoded pass-count)
4. **Builds** — `npx wxt build` (Chrome MV3) + `npx wxt build -b firefox`; both must succeed
5. **§11.4.38 artifact asset verification** (anti-bluff core — see below)
6. **Packaging** — `npm run zip` + `npm run zip:firefox`; each produced `.zip` must exist and exceed a minimum-size floor

Prerequisites

- Run after `npm install` (a populated `node_modules`).
- Tools on PATH: `node`, `npx`, `jq`.
- `npm` scripts present in `package.json`: `lint`, `zip`, `zip:firefox` (all pre-existing).

Usage

```
cd extension && bash ci-ext.sh
```

The script cds to its own directory first, so it is invocable from any cwd. No CLI arguments, no env-var configuration.

Exit codes: 0 all steps PASS (prints CI-EXT: PASS) · 1 first failing step (prints CI-EXT: FAIL (step N) to stderr).

§11.4.38 artifact asset verification (anti-bluff core)

Step 5 is the §11.4.38 “installable-asset evidence” mandate made mechanical: a build can exit 0 and still ship a **stripped** / **missing** / **empty** asset. This step OPENS the produced artifact (it does NOT grep source) and proves every declared user-visible asset is present and non-degenerate:

- jq reads `.output/chrome-mv3/manifest.json` and asserts `manifest_version == 3` and `background.service_worker` is declared.
- It enumerates every asset the manifest declares — icons, the service worker, content-script js/css, the action popup, the options page, web-accessible-resources, and (per Chrome’s rule) the `_locales/<default_locale>/messages.json` required whenever `default_locale` is set with `__MSG_*__` placeholders.
- For each referenced HTML page (popup/options) it also parses the page’s local `src=“...” / href=“...”` references (chunks/assets/icons), skipping `http(s)://data:/mailto:/#` refs.
- Every collected path is stat-checked: it must EXIST on disk and be NON-ZERO. Any missing or empty asset FAILs the step.

This is genuinely anti-bluff: removing an icon from `.output`, or zeroing a content-script chunk, makes step 5 FAIL (verified during authoring).

Edge cases

- **Concurrent checkout mutation (§11.4.84):** transient files (e.g. a leftover `src/__leak_probe__.ts`) or in-flight test fixes in the working tree can flip steps 1–2 between runs. Re-run on a quiescent tree for a stable verdict.
- **_locales packaging:** if `default_locale` is set in the manifest but the build does not emit `_locales/<locale>/messages.json` (WXT copies

`_locales` only from the **public** dir, e.g. `src/public/_locales/`, not `src/_locales`), step 5 FAILs — correctly, because Chrome rejects such an extension at load.

- **No fake PASS / no silent skip**: a step that cannot run FAILs the gate (required-tool check FAILs hard). The script never converts a failure or a skipped step into a PASS.

Internal behaviour

Pure read/build/package — no git, no commit, no container start. Writes only the `.output/` build + zip artifacts. Runs the full vitest suite offline (no live services). Stays well within host-safety limits.

Related

- `extension/package.json` — lint, zip, zip:firefox scripts the gate invokes.
- `extension/wxt.config.ts` — manifest generator (build input).
- Constitution §11.4.38 — installable-asset evidence mandate (step 5).
- Constitution §11.4.18 — script documentation mandate (this companion doc).
- Constitution Hard Stop §1 / project CLAUDE.md — CI is manual, no pipelines.

Last verified: 2026-06-10 — steps 1–6 PASS, step 7 (§11.4.38) correctly FAILs on the genuine missing `_locales/en/messages.json` in the produced Chrome artifact (a real packaging defect outside this script's scope to fix).